Predicting Kit Cycle Time
- Dell Global Operations

From Order Signal to Operational Early Warning

By Samridh Srivastava

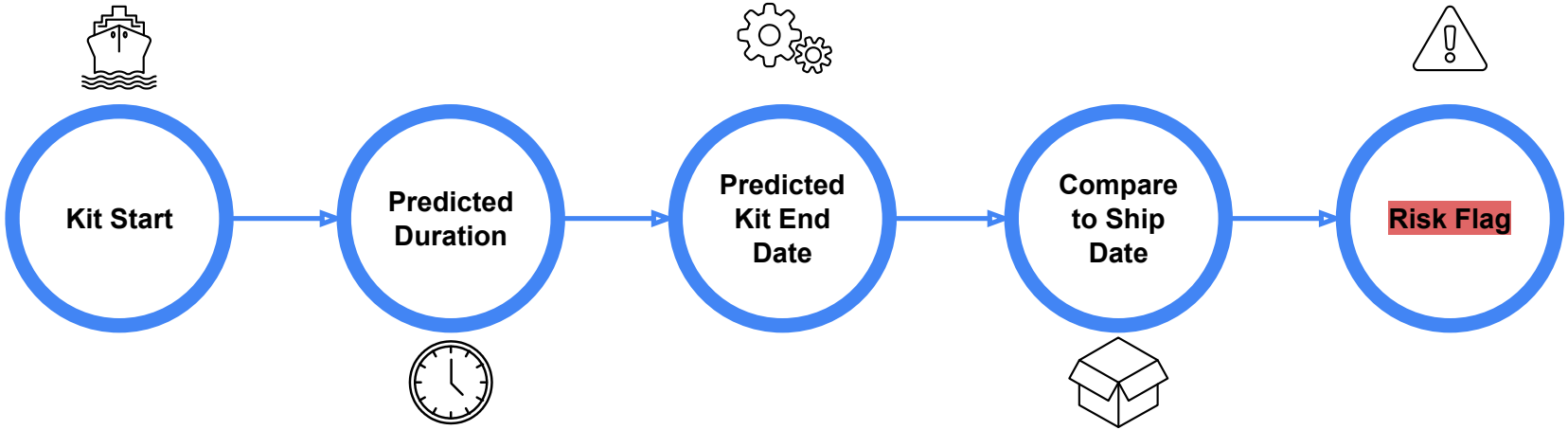
Problem Statement 📌

In **Dell**’s manufacturing workflow, **the *kit phase*** is the critical step where all required components for a sales order are gathered and prepared before assembly begins. Delays in this phase directly impact production schedules and delivery timelines.

The objective of this project is to **predict the expected kit completion date (kit end date)** for each sales order using historical order and operational data.

Can we predict kit cycle time at the moment a kit starts, early enough to intervene?

Inference Pipeline

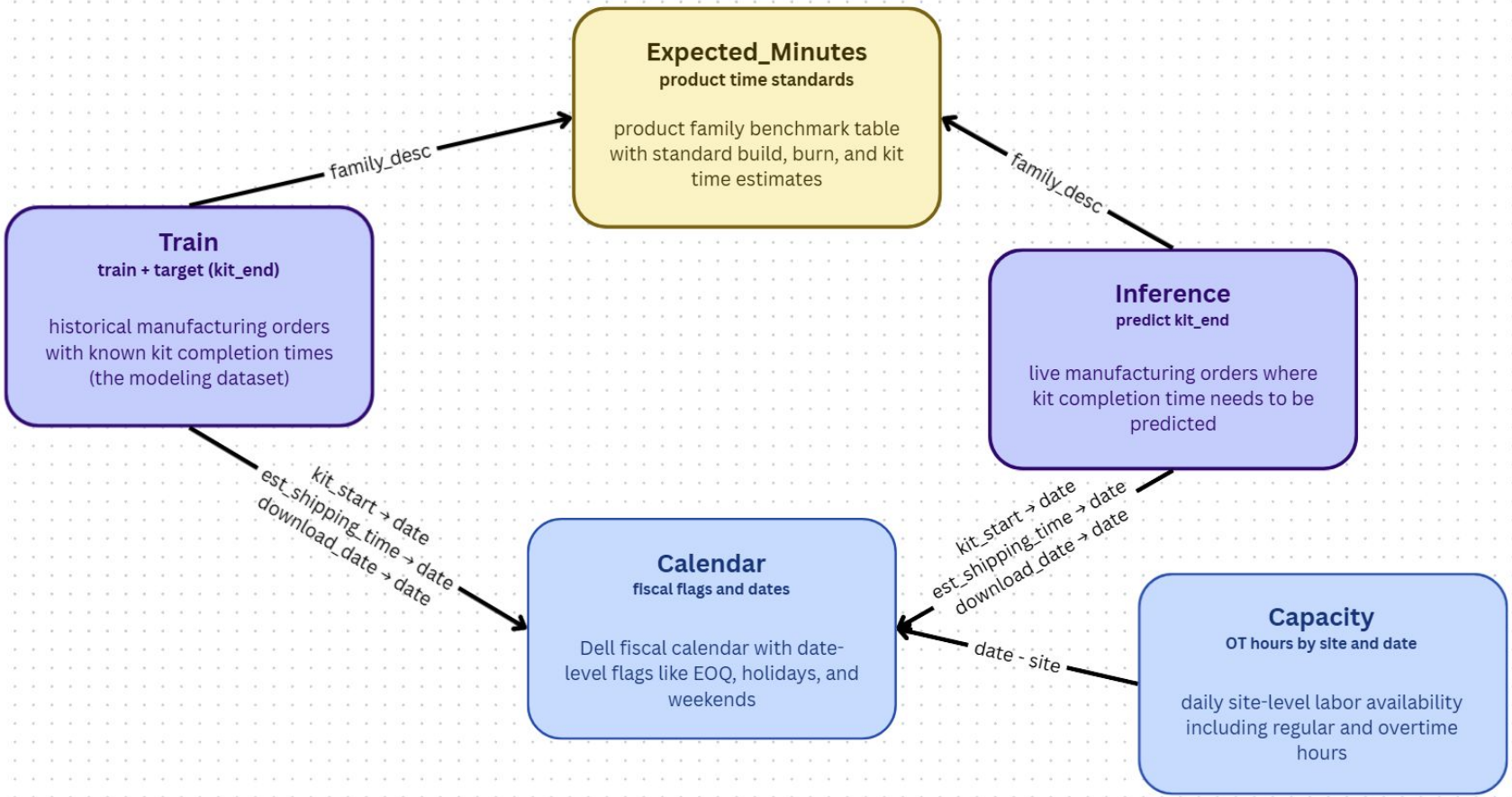


Data Landscape

- **Key stats:** ~180K orders, 360 product families, spanning approx. March 2025 to January 2026.
- **19.5%** of records had **zero-duration** entries (likely system artifacts). After cleaning, **~142K** records remained
- **4 datasets** integrated into a unified analytical view

~19.5% of records with zero duration were removed, as they likely represent auto-closed or cancelled orders (where start and end times are identical), not actual processing time.

Data Landscape - Here's how these four datasets connect to form the analytical view.



Approach & Feature Strategy

The model improves step-by-step by adding real-world operational signals.

Layer	What it uses?	Why it matters?
Baseline	Order + Product details	Captures basic product complexity
Calendar	Date & timing info	Explains when the work happens
Capacity	Workload & system load	Shows how stressed operations are

Note: *All features are available at the start of the kit process, so predictions can be used in real time.*

Baseline*: family_desc, product_group, BUID, MCID, IDC, order_amount, expected_kit_minutes, is_cfi, cfs_flag.*
Calendar*: is_weekend, day_of_week, fiscal_qtr_weeknum, is_eoq.*
Capacity: *ot_utilisation, site_congestion, rolling_7d_mean_duration, prev_day_avg_duration.*

Engineered Features

Features	Definition	Source
Site congestion	Weighted count of concurrent active orders at the same site on kit_start date, weighted by remaining **expected_kit_minutes**	Train + Expected_Minutes
Recent cycle time trend	**Rolling 7-day mean kit duration** at the site level, lagged by 1 day	Train (aggregated)
Prev day avg duration	**Mean kit duration** at the site for the previous calendar day	Train (aggregated)
OT utilisation	**ot_hours / (regular_hours + ot_hours)** for that site-date	Capacity

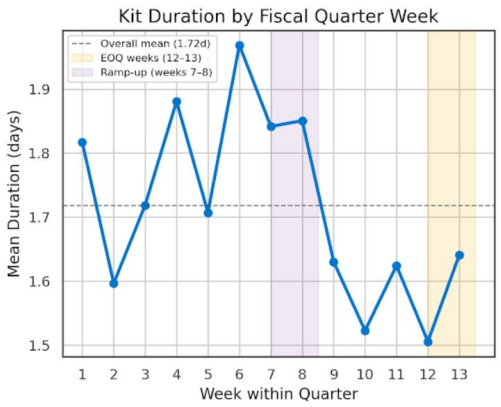
All engineered features use only information available at or before kit_start. Lag features use a 1-day offset to prevent same-day leakage.

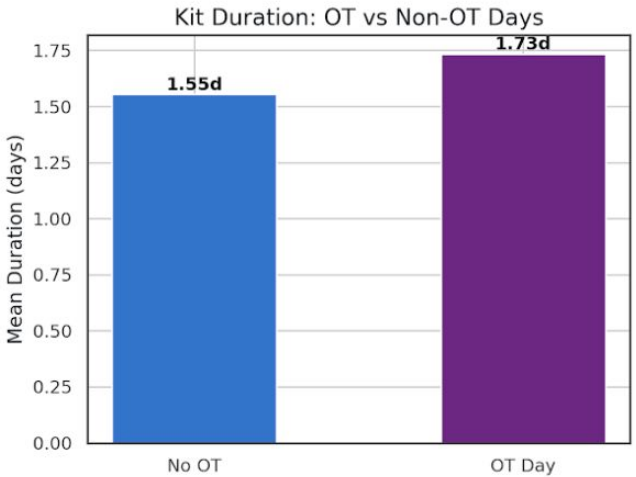
Key Patterns in the Data

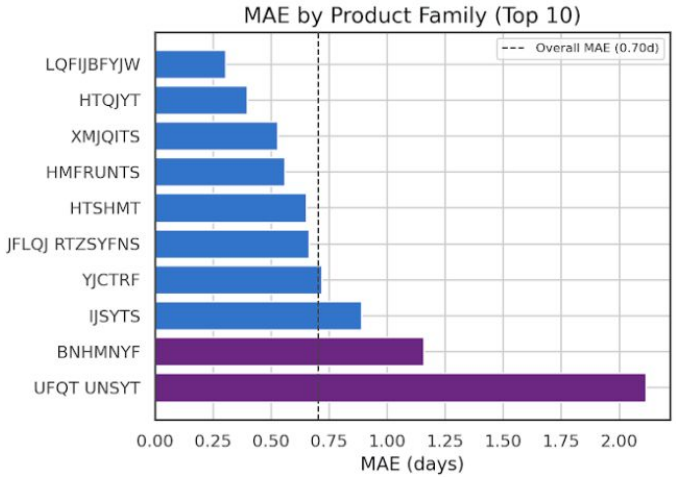
- Mid-Quarter Spike
  - Cycle time peaks at week 6 (~1.97d), 15% above mean, indicating a mid-quarter load surge.
- Capacity Stress
  - Overtime is associated with longer cycle times, suggesting that scaling is reactive rather than planned.
- Product Complexity
  - A small group of product families drives most of the delays and prediction errors, showing that complexity is uneven across products

*Cycle time is driven by **timing**, **system stress**, and **product complexity***

Key Patterns in the Data

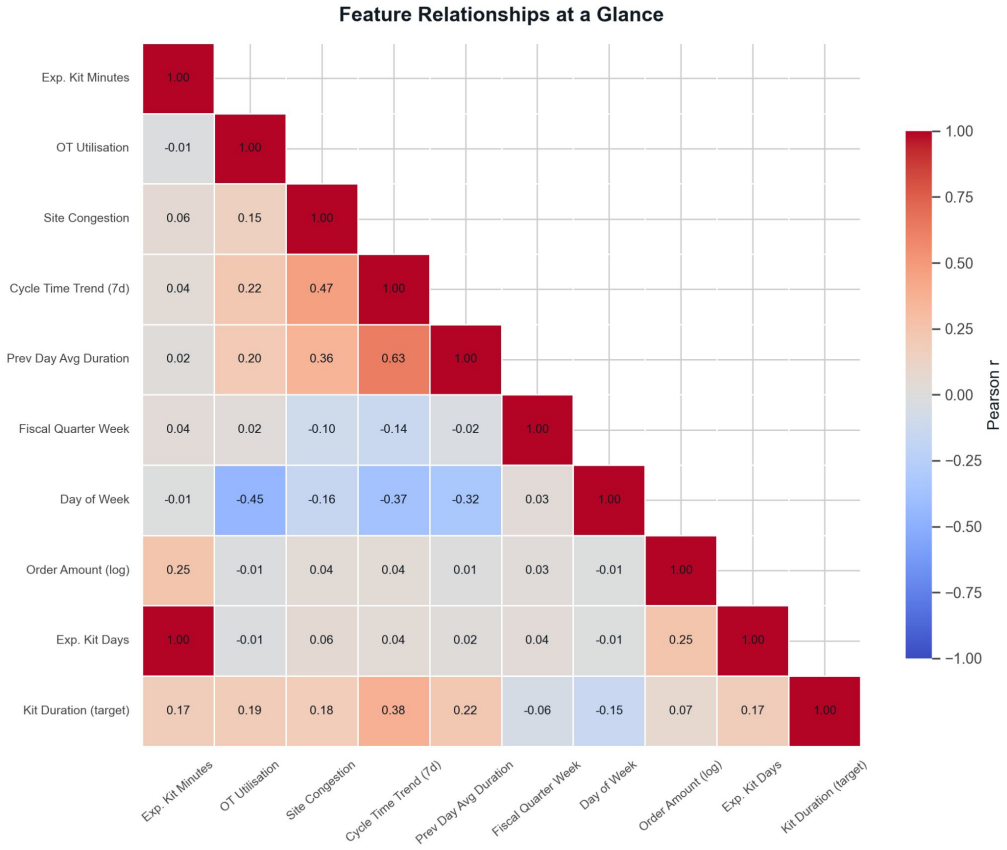






MAE (Mean Absolute Error) *is the average difference between predicted and actual kit completion times, showing how many days the model is off on average.*
Mean duration *is the average time it takes to complete the kit process across all orders.*

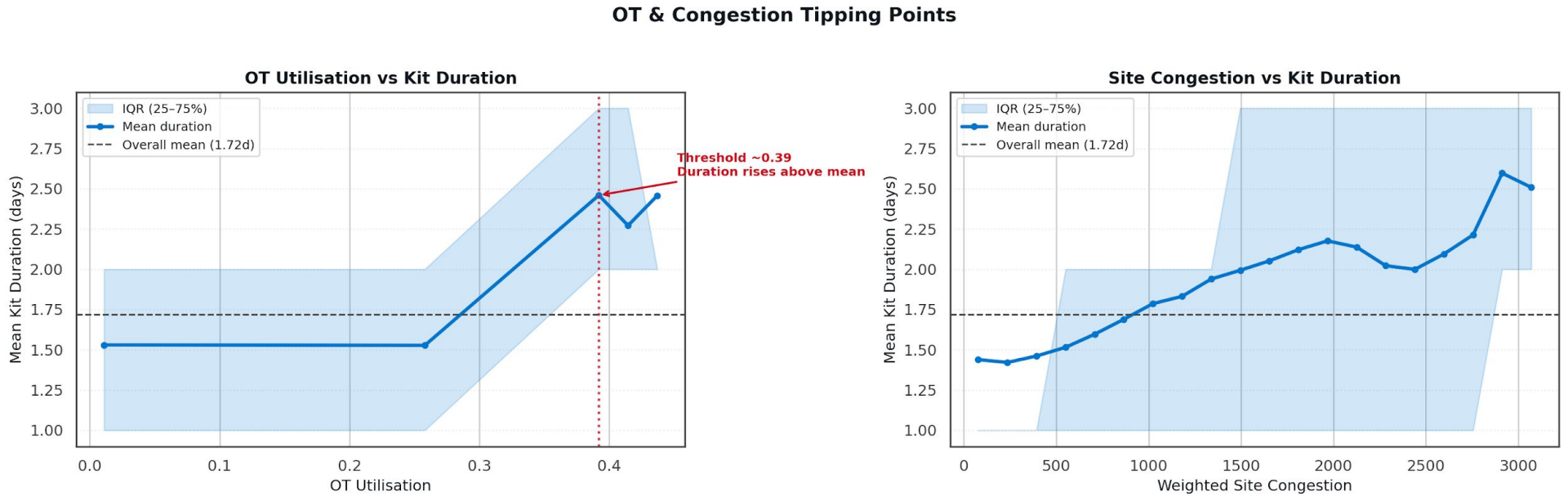
Deep-Dive Exploration



- **Cycle time trend (7d)** has the strongest link with duration (~0.38), showing recent performance matters most
- **Prev day duration** has a positive effect (~0.22), reflecting short-term carryover
- **Site congestion** and **OT utilization** have a moderate positive impact, with higher load increasing delays
- **Day of week** shows a negative effect (~-0.15), meaning some days are consistently faster or slower
- **Order amount** has weak correlation, so size alone doesn't drive duration much
- **Expected kit metrics** (minutes/days) align with duration, making them useful but not sufficient

Most features have **low–moderate correlation**, so multiple factors together drive outcomes

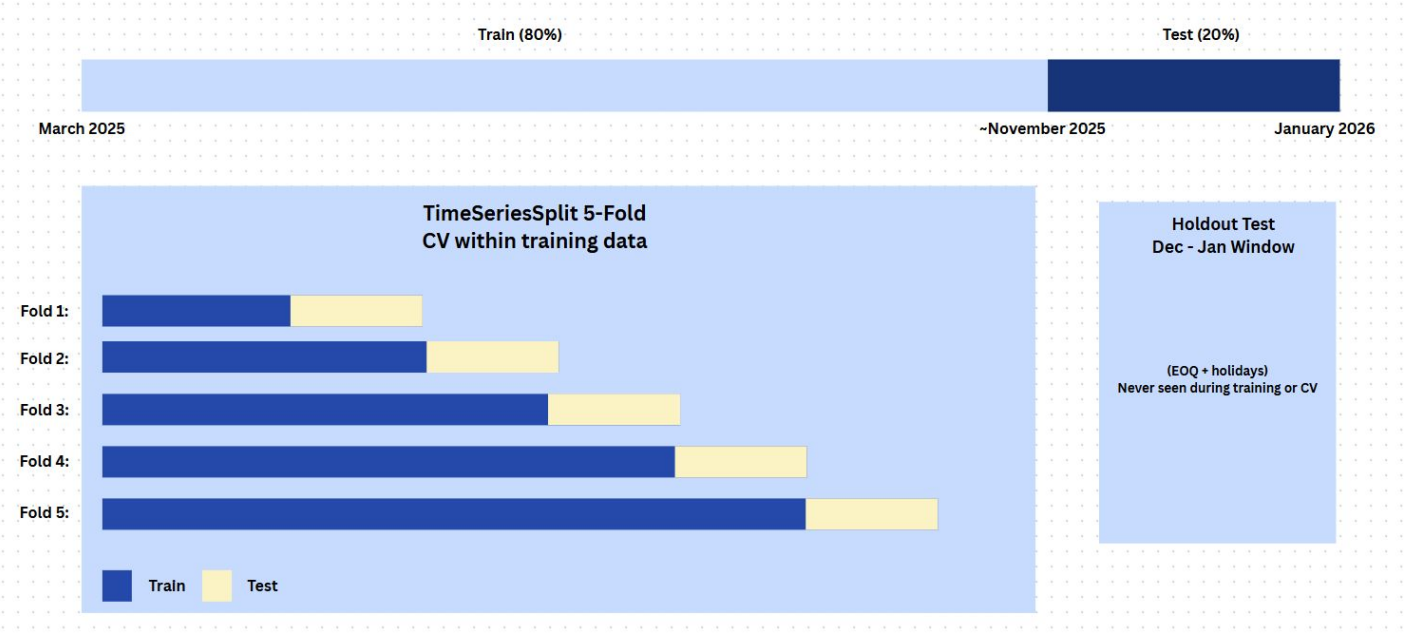
Deep-Dive Exploration



Kit duration remains stable at low OT and congestion levels but increases sharply beyond key thresholds (~0.39 OT and high congestion), indicating clear tipping points where system performance degrades.

Tipping point at ~0.39 OT utilisation identified via binned mean analysis (10 equal-frequency bins)

Validation Strategy



Temporal integrity ensured

- **TimeSeriesSplit (5 folds)**: train on past and validate on next window
- **No random K-Fold** (prevents future leakage)

Leakage-free features

- All inputs available at kit_start
- No use of kit_end (target)

Stress-test evaluation

- Holdout: Dec 2025 – Jan 2026
- EOQ + holidays = worst-case scenario

No Future Data Leakage

Model Performance

Model	Features	Test MAE
Ridge (Baseline)	Order + Expected Minutes	0.92 days (22 hrs)
XGBoost	Order + Expected Minutes	0.87 days (21 hrs)
XGBoost	Above Features + Calendar	0.75 days (18 hrs)
XGBoost	Above Features + Capacity	0.70 days (16.86 hrs)
XGBoost	Above Features + Capacity (tuned hyperparameters)	0.70 days (16.89 hrs)

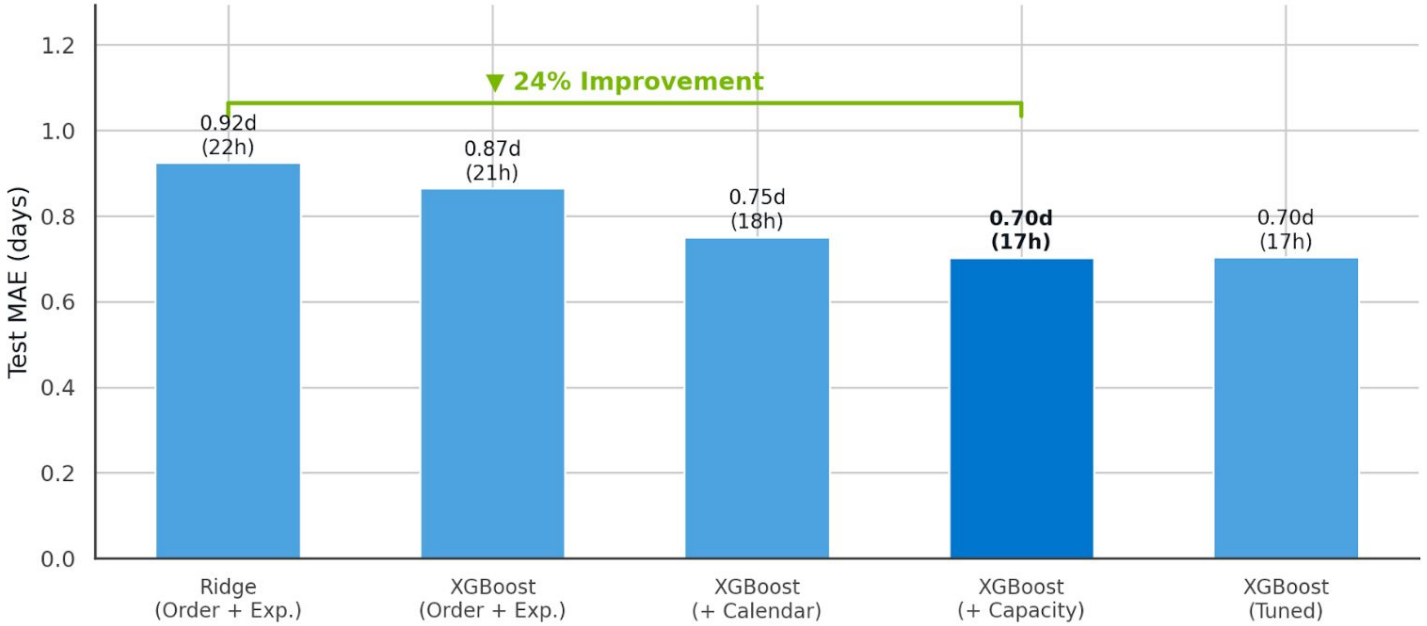
Hyperparameter tuning via TimeSeriesSplit grid search produced <0.005 day improvement, confirming that feature engineering, not model tuning, drove the 24% gain

Model Performance

Evaluation: MAE reported on temporal holdout (Dec 2025 - Jan 2026); CV MAE available in notebook.

Tuning: Grid search with TimeSeriesSplit (5 folds).
Search space: max_depth {4,6,8}, learning_rate {0.01,0.05,0.1}, n_estimators {500}.
Best parameter: max_depth=4, learning_rate=0.01, n_estimators=500, subsample=0.7, colsample_bytree=0.7.
Improvement (<0.005 days) based on this search.

Model Performance



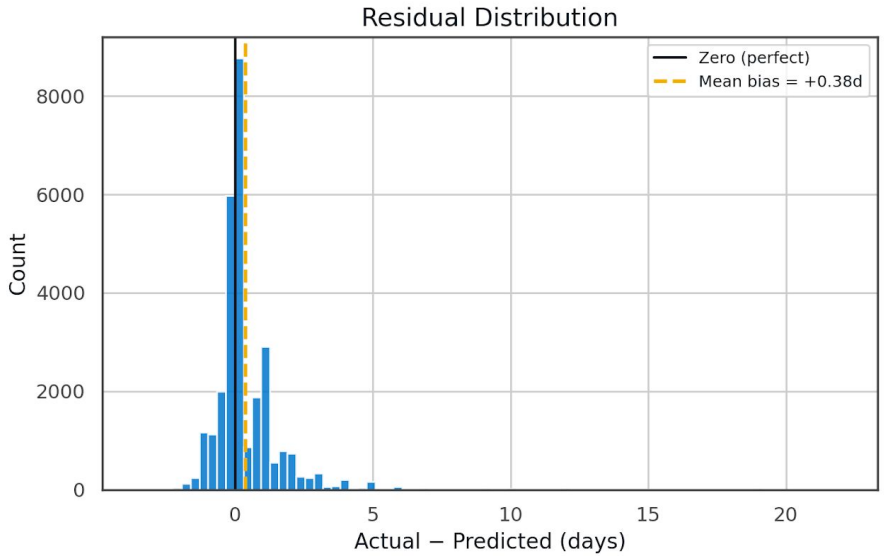
Where the Model Struggles

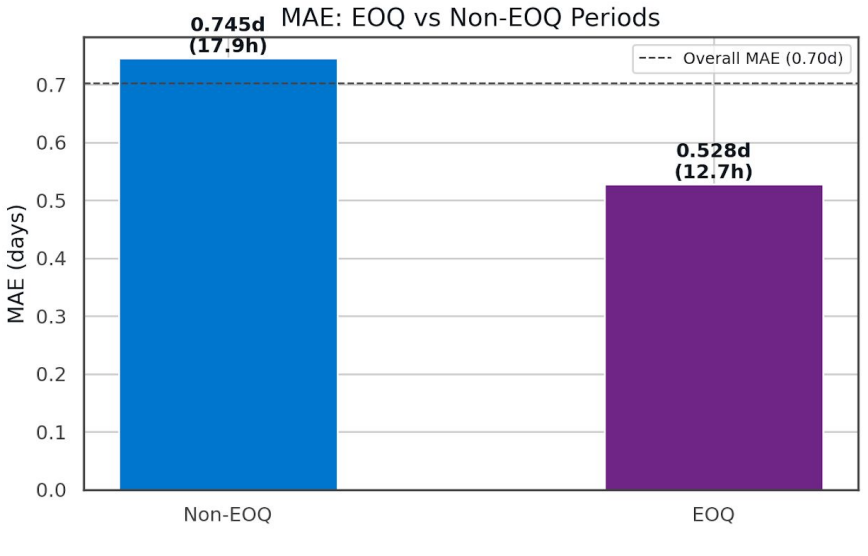
- The model under-predicts by ~0.38 days on average (it's optimistic)
- Mid-quarter weeks (6–8) drive the bulk of under-prediction. The model struggles most with the demand spike before EOQ, not at EOQ itself
- A few product families (UFQT UNSYT) have 3× the average error

The test window covers EOQ + holidays. It is a deliberately challenging period. The bias tells us the model needs calibration for high-stress periods

Where the Model Struggles

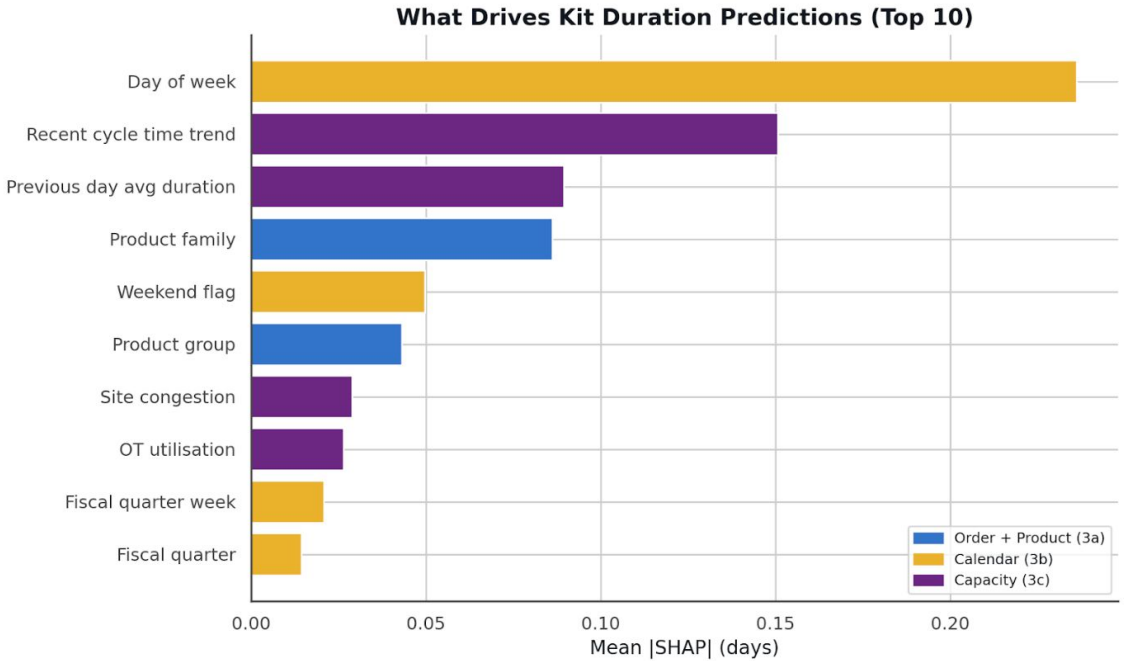
Model Limitations: Optimistic Bias & Mid-Quarter Spike Underfit





Bias +0.38 days (Dec - Jan holdout only; EOQ/holiday stress period, not year-round).

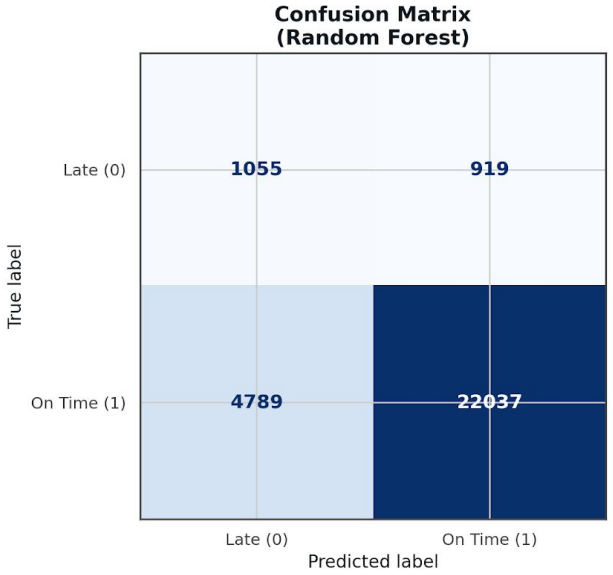
What Drives Predictions?

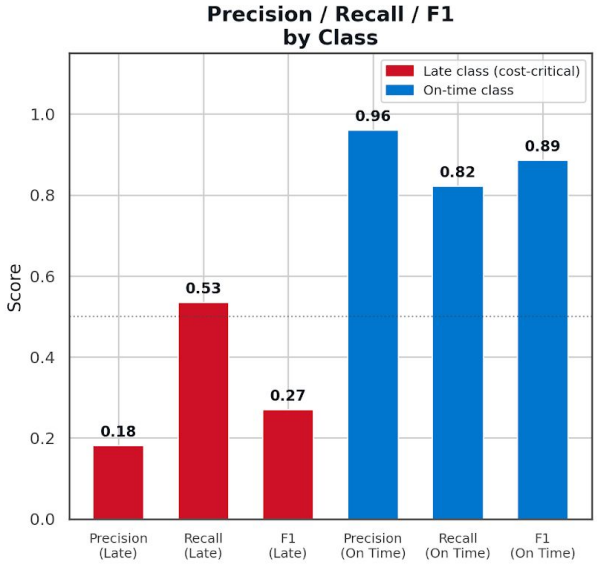


What this tells Operations

- **Weekend effect is dominant**
  Fri/Sat starts spill into weekends → ~+0.5 days
  → Staffing gaps are the biggest driver of delays
- **Cycle time has momentum**
  Slow last 7 days → next kits stay slow
  → Backlogs persist, don't reset overnight
- **Yesterday predicts today**
  Previous day's performance is the strongest short-term signal
  → Daily floor efficiency carries forward

On-Time Risk Flag Performance





Model Summary

ROC-AUC	**0.731**
PR-AUC	**0.967**
Total late orders (test)	**1,974**
Caught by flag	**1,055 (53%)**
Missed by flag (FN)	**919 (47%)**

⚠ **Of 1,974 orders that were actually late, 919 were missed by the flag (47%).**

These are the highest-cost errors — orders that ship late without warning.

The model captures 53% of late orders but misses 47%, meaning a significant number of high-cost late shipments still go unflagged despite strong overall performance.

Late = predicted kit_end > est_ship_date; same features as regression; threshold = 0.5

Operational Recommendations

R1

Deploy an on-time risk flag at kit initiation

1. The model scores every order at **kit_start** and flags **HIGH / MEDIUM / LOW** risk.
2. **Inference data**: **433 HIGH**-risk orders identified out of 12,068 (3.6% of volume), enabling targeted intervention.

R2

Build mid-quarter capacity buffers

On-time rates drop during weeks 6–8. Pre-scaling capacity and pre-staging inventory 1–2 weeks in advance eliminates reactive delays and reduces peak congestion

R3

Refresh benchmarks for high-error product families

1. Segment analysis shows **certain high-volume families** (e.g., UFQT UNSYT) with **~3× higher error**, indicating stale or incomplete benchmarks.
2. Audit top high-error families against trailing **12-month actuals** and update benchmarks **where gaps exceed 15%**; investigate unmodelled constraints.
3. **Expected impact**: ~1+ day improvement in prediction accuracy for affected families, leading to more reliable delivery commitments.

Operational Recommendations

R4

Congestion-aware kit release gates

Hold kit releases when factory load exceeds a threshold, it reduces the right tail of cycle time

R5

OT utilisation as a leading indicator dashboard

1. **OT utilisation** is a strong real-time proxy for system stress and rising delay risk.
2. Build a daily dashboard with OT gauge, 7-day trend, and at-risk order count under sustained OT levels.
3. **Expected impact:** enables a 24–72 hour early intervention window, allowing proactive capacity and scheduling decisions.

Operational Recommendations

If expedited shipping costs $50–$150 per late order and the Risk Flag prevents even 30% of the 433 high-risk orders from being late, that's $6.5K - $19.5K saved per batch, and the savings grow directly with order volume.

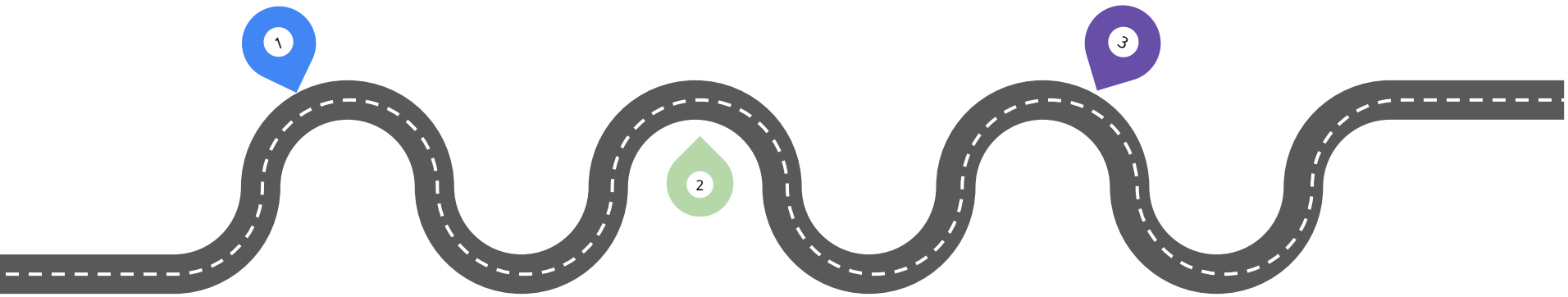
Note: Illustrative only to show impact. Actual savings depend on Dell's expedite cost structure and intervention success rate

Implementation Roadmap

🟢- Easy effort
🟡- Medium effort
🔴- Hard effort

Deploy Risk Flag (R1) 🟢 + EOQ Protocol (R2) 🟢

OT dashboard (R5) 🔴 + model retraining pipeline



Congestion gate (R4) 🟢 + benchmark audit (R3) 🟡

Summary & Key Takeaways

1. Kit cycle time is predictable within 17 hours using signals available at kit initiation
2. Feature engineering drove 24% improvement, not model complexity
3. The model translates directly into an operational early-warning system with five concrete deployment paths